LECTURE 5

Events + Validation

Xử lý Sự kiện • Xác thực Dữ liệu Form • Phản hồi Người dùng (Toast, Snackbar, Dialog)

Đỗ Phú Huy
BM CNTT - Khoa CNS - ĐH SPKT ĐN

Lecture 5 • Events + Validation

Mục tiêu & Lộ trình bài học

1. Cơ chế Lắng nghe & Xử lý Sự kiện (Event Listeners & ViewBinding)
2. Kiểm tra & Xác thực Dữ liệu Form (Form Validation & TextInputLayout)
3. Phản hồi & Thông báo Người dùng (Toast, Snackbar, AlertDialog)
4. Thực hành Dự án: Interactive Registration Form hoàn chỉnh

Kiến trúc Hướng Sự kiện (Event-driven Architecture)

Android UI vận hành hoàn toàn dựa trên các sự kiện tương tác của người dùng

Nguyên lý cốt lõi của Android UI:

• Giao diện Android tĩnh cho đến khi có một sự kiện (Event) xảy ra.

• Sự kiện có thể đến từ người dùng (chạm nút, gõ phím, vuốt) hoặc từ hệ thống (nhận cuộc gọi, cắm sạc, đổi hướng xoay).

• Khi sự kiện phát sinh, một đối tượng Event được đưa vào hàng đợi MessageQueue của Main Thread.

Chu trình Xử lý Sự kiện 4 bước:

• 1. User Action: Người dùng thực hiện thao tác vật lý trên màn hình cảm ứng.

• 2. Event Generation: Hệ điều hành Android phát hiện tọa độ và chuyển thành MotionEvent / KeyEvent.

• 3. Listener Dispatch: View tương ứng nhận sự kiện và gọi hàm Callback đã đăng ký.

• 4. State Update & UI Render: Lập trình viên cập nhật dữ liệu và giao diện phản ánh kết quả mới.

Sơ đồ Luồng Xử lý Sự kiện trong Android

Mô hình tương tác trực quan từ thao tác người dùng đến khi giao diện cập nhật



Phần 1

Cơ chế Lắng nghe & Xử lý Sự kiện

Event Listeners, ViewBinding và các kỹ thuật bắt sự kiện click, text change, checked change

Bắt sự kiện Click: Các cách tiếp cận phổ biến

So sánh cú pháp giữa Anonymous Class truyền thống và SAM Conversion Lambda hiện đại

CÁCH 1: SAM CONVERSION LAMBDA (KHUYÊN DÙNG)

• Cú pháp cực kỳ ngắn gọn, chuẩn mực Kotlin:

```
binding.btnSubmit.setOnClickListener {
    val email = binding.etEmail.text.toString()
    validateAndSubmit(email)
}
```

• Kotlin tự động chuyển đổi Single Abstract Method (SAM) thành cú pháp lambda gọn gàng.

• Không cần khai báo tường minh kiểu đối tượng lặp lại.

CÁCH 2: ANONYMOUS OBJECT (CỔ ĐIỂN)

• Cú pháp Java truyền thống chuyển dịch sang Kotlin:

```
binding.btnSubmit.setOnClickListener(object :
    View.OnClickListener {
    override fun onClick(v: View?) {
        validateAndSubmit()
    }
})
```

• Dài dòng, nhiều mã boilerplate không cần thiết.

• Chỉ dùng khi cần truyền đối tượng Listener vào nhiều View cùng lúc.

So sánh: android:onClick (XML) vs Kotlin Listener

Tại sao Google khuyến nghị loại bỏ thuộc tính onClick trong XML?

KHAI BÁO XML: android:onClick (KHÔNG NÊN DÙNG)

• Khai báo trực tiếp trong XML:

android:onClick="onSaveClick"

• Activity phải có hàm trùng tên:

fun onSaveClick(v: View)

NHƯỢC ĐIỂM NGHIÊM TRỌNG:

• Sử dụng Reflection chạy ngầm -> Giảm hiệu năng

• Không kiểm tra kiểu tại thời điểm biên dịch (Compile-time)

• Đổi tên hàm trong Kotlin nhưng quên sửa XML -> Crash app!

• Không hoạt động bên trong Fragment!

KOTLIN LISTENER: setOnClickListener (CHUẨN MỰC)

• Khai báo trực tiếp trong code Kotlin thông qua ViewBinding.

ƯU ĐIỂM VƯỢT TRỘI:

• Hoàn toàn an toàn kiểu dữ liệu (Type-safe, Null-safe)

• Hỗ trợ Refactor / Đổi tên hàm tự động trên toàn dự án

• Dễ dàng viết Unit Test và gỡ lỗi (Debugging)

• Hoạt động đồng nhất trong cả Activity, Fragment và Custom View.

Lắng nghe Thay đổi Văn bản trong EditText

Theo dõi người dùng gõ phím để kiểm tra dữ liệu theo thời gian thực (Real-time)

Giao diện TextWatcher truyền thống:

- Yêu cầu implement cả 3 hàm callback: beforeTextChanged(), onTextChanged(), và afterTextChanged().
- Trong 90% trường hợp thực tế, lập trình viên chỉ cần lấy chuỗi ký tự sau khi người dùng vừa gõ xong.

KTX Extension: doOnTextChanged & doAfterTextChanged (Hiện đại):

- Thư viện 'androidx.core:core-ktx' cung cấp các hàm mở rộng trực tiếp cho EditText.
- Chỉ cần viết 1 lambda duy nhất, loại bỏ hoàn toàn mã thừa:

```
binding.etEmail.doAfterTextChanged { text ->
    validateEmailRealtime(text.toString())
}
```

Mã nguồn: Lắng nghe Gõ phím với doAfterTextChanged

Triển khai kiểm tra tức thời và xóa thông báo lỗi khi người dùng sửa dữ liệu

```
// File: MainActivity.kt
import androidx.core.widget.doAfterTextChanged

class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        // Lắng nghe sự kiện gõ phím trên ô Email
        binding.etEmail.doAfterTextChanged { editable ->
            val input = editable?.toString()?.trim().orEmpty()
            if (input.isNotEmpty()) {
                // Xóa lỗi ngay khi người dùng bắt đầu nhập lại
                binding.tilEmail.error = null
            }
        }
    }
}
```

Điểm cốt lõi:

- Hàm 'doAfterTextChanged' nhận tham số 'Editable?' chứa nội dung sau khi gõ.
- Dùng 'editable?.toString()?.trim().orEmpty()' để bảo đảm an toàn null tuyệt đối.
- Khi người dùng gõ ký tự mới, chủ động gán 'error = null' để tắt viền đỏ cảnh báo của TextInputLayout.
- Giúp giao diện phản hồi mượt mà, chuyên nghiệp.

Xử lý Sự kiện Lựa chọn: CheckBox & RadioGroup

Lắng nghe trạng thái bật/tắt (Boolean) và chọn một trong nhiều (Single Choice)

```
// 1. Lắng nghe CheckBox / Switch
binding.cbTerms.setOnCheckedChangeListener { _, isChecked ->
    // Kích hoạt nút Submit chỉ khi đồng ý điều khoản
    binding.btnRegister.isEnabled = isChecked
}

// 2. Lắng nghe RadioGroup (Chọn giới tính)
binding.rgGender.setOnCheckedChangeListener { _, checkedId ->
    val selectedGender = when (checkedId) {
        R.id.rbMale -> "Nam"
        R.id.rbFemale -> "Nữ"
        else -> "Khác"
    }
    Log.d("APP_LOG", "Giới tính: $selectedGender")
}
```

Lưu ý lập trình:

- CheckBox / Switch: Callback truyền tham số 'isChecked: Boolean'.
- Thường dùng để bật/tắt nút Đăng ký hoặc hiển thị thêm phần nhập phụ.
- RadioGroup: Callback truyền 'checkedId: Int' tương ứng ID của RadioButton được chọn trong XML.
- Dùng biểu thức 'when' để ánh xạ ID sang giá trị dữ liệu tương ứng.

Bảng Tra Cứu: Các Listener Phổ Biến trong Android UI

Tổng hợp các giao diện lắng nghe và hàm callback tương ứng theo từng loại View

SỰ KIỆN CLICK & VĂN BẢN

• Nút bấm / View bất kỳ:
- Interface: View.OnClickListener
- Hàm: onClick(v: View)
- Cú pháp: view.setOnClickListener { ... }

• Nhấn giữ lâu (Long Click):
- Interface: View.OnLongClickListener
- Hàm: onLongClick(v: View): Boolean
- Cú pháp: view.setOnLongClickListener { true }

• Thay đổi chữ trong EditText:
- KTX Method: doAfterTextChanged { text -> ... }

SỰ KIỆN LỰA CHỌN & CHUYỂN MÀN

• CheckBox / Switch bật tắt:
- Interface: CompoundButton.OnCheckedChangeListener
- Hàm: onCheckedChanged(buttonView, isChecked)

• RadioGroup chọn 1 trong nhiều:
- Interface: RadioGroup.OnCheckedChangeListener
- Hàm: onCheckedChanged(group, checkedId)

• Mất/Nhận tiêu điểm nhập liệu (Focus):
- Interface: View.OnFocusChangeListener
- Hàm: onFocusChange(v, hasFocus)

Phần 2

Kiểm tra & Xác thực Dữ liệu Form

- Kỹ thuật Validate dữ liệu đầu vào
- Biểu thức chính quy Regex
- TextInputLayout
- Thiết kế Clean Validation

Tại sao Xác thực dữ liệu là Bắt Buộc?

Nguyên tắc cốt lõi: Không bao giờ tin tưởng tuyệt đối dữ liệu do người dùng nhập vào

3 Trụ cột của Xác thực Dữ liệu (Validation):

1. Tính Toàn vẹn dữ liệu (Data Integrity): Đảm bảo dữ liệu đúng kiểu, đúng độ dài và cấu trúc trước khi lưu vào Database hoặc gửi lên Server qua REST API.

2. Trải nghiệm người dùng (UX Feedback): Giúp người dùng biết ngay mình sai ở đâu, vì sao sai và cần sửa như thế nào thay vì để hệ thống báo lỗi mù mờ.

3. Chống Crash ứng dụng: Ngăn ngừa các lỗi nghiêm trọng như NumberFormatException hoặc NullPointerException.

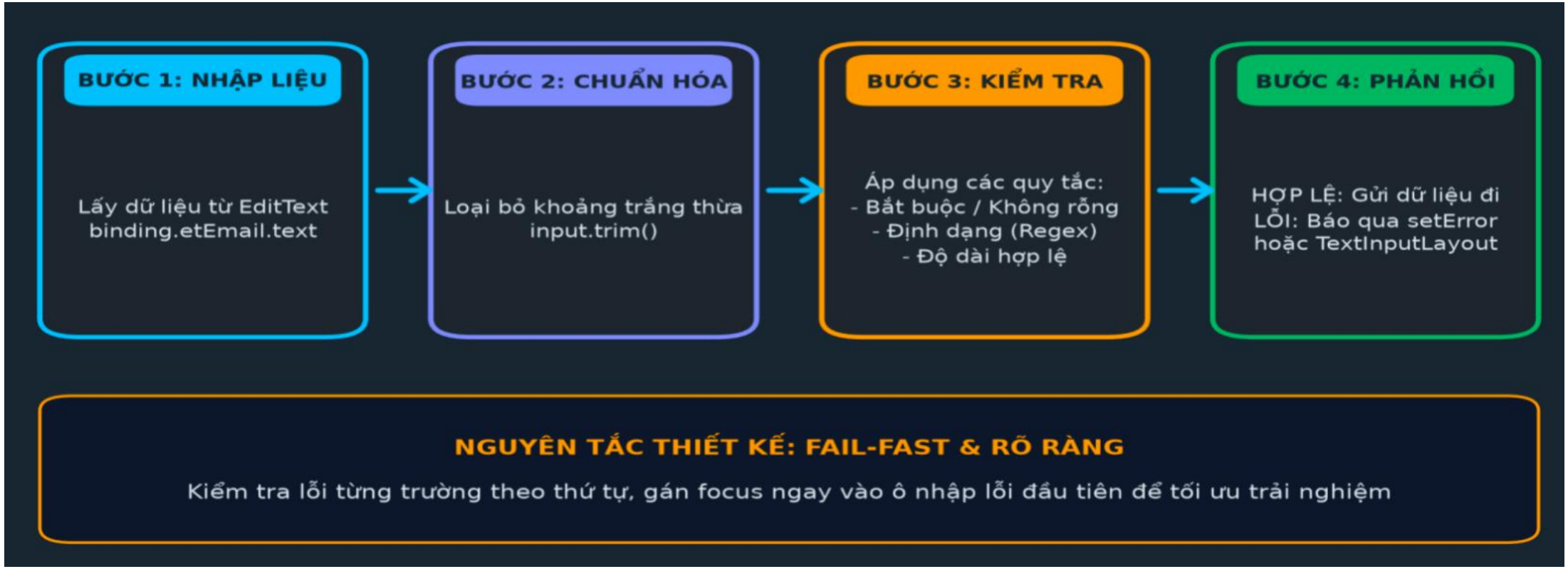
Nguyên tắc Thiết kế Fail-Fast:

• Kiểm tra ngay tại tầng UI (Client-side validation) để tiết kiệm băng thông mạng và giảm tải cho máy chủ.

• Thông báo lỗi gần nhất với vị trí ô nhập liệu bị sai.

Sơ đồ Quy trình xác thực Form nhiều bước

Luồng xử lý từ lúc người dùng bấm nút Gửi đến khi dữ liệu được đóng gói an toàn



Kỹ thuật Validate Cơ bản: Kiểm tra rỗng, khoảng trắng & độ dài

Tận dụng các hàm chuẩn mực được xây dựng sẵn trong thư viện Kotlin Standard Library

1. Phân biệt isEmpty() vs isBlank():

• text.isEmpty(): Kiểm tra độ dài có bằng 0 hay không. Chuỗi chứa dấu cách " " sẽ trả về false!

• text.isBlank(): Kiểm tra xem chuỗi có rỗng hoặc chỉ chứa toàn khoảng trắng hay không (Khuyên dùng!).

• text.isNullOrBlank(): An toàn tuyệt đối với biến có thể null. Trả về true nếu null, rỗng hoặc toàn khoảng trắng.

2. Kiểm tra Độ dài và Miền giá trị Số:

• Kiểm tra độ dài mật khẩu: if (password.length < 6) { báo lỗi }

• Chuyển đổi số an toàn với toIntOrNull() hoặc toDoubleOrNull():

```
val age = binding.etAge.text.toString().toIntOrNull()
if (age == null || age !in 18..100) { báo lỗi tuổi không hợp lệ }
```

Validate Nâng cao: Biểu thức Chính quy (Regex)

Kiểm tra các mẫu định dạng phức tạp như Email, Số điện thoại và Mật khẩu mạnh

ĐỊNH DẠNG EMAIL CHUẨN

- **Cách 1: Dùng Patterns chuẩn của Android:**

```
val isEmailValid =
android.util.Patterns.EMAIL_ADDRESS.matcher(ema
il).matches()
```

- **Cách 2: Tự định nghĩa Regex Pattern:**

```
val emailRegex = "^[A-Za-z0-9+_.-
]+@(.+)$".toRegex()
 if (!email.matches(emailRegex)) {
    // Báo lỗi định dạng email
 }
```

SỐ ĐIỆN THOẠI VIỆT NAM & MẬT KHẨU

- **Số điện thoại Việt Nam (10 số, bắt đầu bằng 0):**

```
val phoneRegex = "^(03|05|07|08|09)\\d{8}$".toRegex()
val isPhoneValid = phone.matches(phoneRegex)
```

- **Mật khẩu mạnh (Ít nhất 8 ký tự, có số & chữ in hoa):**

```
val passRegex = "^(?=.*[A-Z])(?=.*\\d).{8,}$".toRegex()
val isPassValid = password.matches(passRegex)
```

Hiển thị Lỗi: EditText.setError vs TextInputLayout

Nâng cấp giao diện từ cảnh báo popup cũ kỹ sang chuẩn thiết kế Material Design 3

Hạn chế của EditText.setError() truyền thống:

- Chỉ hiển thị icon chấm than nhỏ và một popup đen che khuất nội dung bên dưới.
- Icon thông báo dễ bị che mất trên màn hình nhỏ hoặc bàn phím ảo đẩy lên.
- Khó tùy biến màu sắc và phông chữ theo phong cách thiết kế của ứng dụng.

Ưu điểm vượt trội của TextInputLayout (Material Components):

- Bọc bên ngoài TextInputEditText để tạo hiệu ứng Floating Label.
- Dòng báo lỗi hiển thị ngay dưới ô nhập với hiệu ứng đổi viền màu đỏ nổi bật.
- Hỗ trợ sẵn nút ẩn/hiện mật khẩu chỉ bằng 1 dòng khai báo XML.

Mã nguồn: Sử dụng TextInputLayout trong XML & Kotlin

Khai báo khung nhập liệu chuẩn Material và kích hoạt báo lỗi linh hoạt

```
<!-- File: res/layout/activity_main.xml -->
<com.google.android.material.textfield.TextInputLayout
    android:id="@+id/tilEmail"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Địa chỉ Email"
    app:errorEnabled="true">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress" />
</com.google.android.material.textfield.TextInputLayout>

// Tại Kotlin: Bật và Tắt lỗi dễ dàng
binding.tilEmail.error = "Email không hợp lệ!" // Bật lỗi
binding.tilEmail.error = null                  // Xóa lỗi
```

Điểm cần ghi nhớ:

- Luôn đặt 'app:errorEnabled="true"' trên TextInputLayout.
- Thuộc tính này giữ sẵn một khoảng không gian dưới đáy, tránh làm layout bị giật khi dòng chữ báo lỗi xuất hiện.
- Trong Kotlin, gán 'error = "Nội dung"' để kích hoạt viền đỏ.
- Gán 'error = null' để tắt viền đỏ và ẩn thông báo lỗi.

Kiến trúc: Tách biệt hàm kiểm tra & Data Class

Nguyên tắc Single Responsibility: Tách logic kiểm tra ra khỏi Activity

```
// 1. Data class đại diện cho kết quả kiểm tra
data class ValidationResult(
    val isValid: Boolean,
    val errorMessage: String? = null
)

// 2. Đối tượng chứa các hàm kiểm tra độc lập
object FormValidator {
    fun validateEmail(email: String): ValidationResult {
        if (email.isBlank()) {
            return ValidationResult(false, "Email không được để trống")
        }
        if
(!android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
            return ValidationResult(false, "Định dạng email sai")
        }
        return ValidationResult(true)
    }
}
```

Lợi ích kiến trúc:

- Tách rời logic nghiệp vụ khỏi giao diện người dùng (UI).
- Dễ dàng viết Unit Test độc lập trên JVM mà không cần chạy máy ảo Android Emulator.
- Có thể tái sử dụng hàm 'validateEmail()' ở màn hình Đăng ký, Đăng nhập và Quên mật khẩu.
- Mã nguồn ngắn gọn, tường minh và dễ bảo trì.

Chiến lược Kiểm tra: Fail-Fast vs Báo lỗi Đồng loạt

Lựa chọn cách tiếp cận phù hợp với từng ngữ cảnh tương tác của người dùng

CHIẾN LƯỢC 1: FAIL-FAST (EARLY RETURN)

- Nguyên tắc: Kiểm tra lần lượt từ trên xuống dưới, gặp lỗi đầu tiên lập tức dừng lại.
- Mã nguồn:

```
if (email.isBlank()) { tilEmail.error = ...; return }
if (pass.isBlank()) { tilPass.error = ...; return }
```

- Ưu điểm: Code cực kỳ đơn giản, gọn gàng.
- Nhược điểm: Người dùng phải bấm Submit nhiều lần để biết hết các lỗi trên form.

CHIẾN LƯỢC 2: CUMULATIVE ERRORS (ĐỒNG LOẠT)

- Nguyên tắc: Kiểm tra toàn bộ các trường, hiển thị viền đỏ đồng loạt trên tất cả các ô lỗi.
- Mã nguồn:

```
var isFormValid = true
if (email.isBlank()) { tilEmail.error = ...; isFormValid = false }
if (pass.isBlank()) { tilPass.error = ...; isFormValid = false }
if (!isFormValid) return
```

- Ưu điểm: Trải nghiệm người dùng tốt nhất!

Phần 3

Phản hồi & Thông báo Người dùng

Toast, Snackbar, AlertDialog
--> Khi nào nên dùng, cách gắn nút hành động Action và quản lý Context an toàn

Phân loại 3 Mức độ Phản hồi trong Android

Chọn đúng kênh thông báo giúp cải thiện đáng kể sự hài lòng của người dùng

1. Mức độ Nhẹ - Thoáng qua: Toast

• Dành cho thông tin bổ sung, không quan trọng, không yêu cầu người dùng chú ý cao.

• Tự động biến mất sau thời gian ngắn (2 - 3.5 giây). Không có nút bấm tương tác.

2. Mức độ Vừa - Có thể Hành động: Snackbar

• Chuẩn mực thông báo hiện đại của Material Design. Xuất hiện ở đáy màn hình.

• Cho phép đính kèm nút hành động (Ví dụ: Thử lại, Xem giỏ hàng, Hoàn tác thao tác xóa).

3. Mức độ Nặng - Yêu cầu Quyết định: AlertDialog

• Hộp thoại bật lên chính giữa màn hình, làm mờ giao diện phía sau và chặn tương tác.

• Bắt buộc người dùng phải xác nhận (Đồng ý / Hủy bỏ) đối với các hành động nguy hiểm.

Sơ đồ So sánh Đặc tính: Toast vs Snackbar vs Dialog

Bảng đối chiếu tiêu chí sử dụng, khả năng tương tác và ngữ cảnh phù hợp

TOAST

- Thời điểm: Thông báo ngắn, thoáng qua
- Vị trí: Nổi phía dưới màn hình
- Tương tác: KHÔNG thể bấm hay tương tác
- Tự biến mất: Có (SHORT: 2s, LONG: 3.5s)
- Context: Dùng được với Application Context
- Ví dụ: "Đã lưu cấu hình", "Không có mạng"

SNACKBAR

- Thời điểm: Thông báo gắn liền hành động
- Vị trí: Đáy màn hình (gần với View cha)
- Tương tác: CÓ NÚT ACTION (Hoàn tác, Thử lại)
- Tự biến mất: Tùy chọn (LENGTH_INDEFINITE)
- Tích hợp: Tự động né FloatingActionButton
- Ví dụ: "Đã xóa sinh viên" [HOÀN TÁC]

ALERT DIALOG

- Thời điểm: Hành động quan trọng / Nguy hiểm
- Vị trí: Chính giữa màn hình, che mờ nền
- Tương tác: BẮT BUỘC người dùng chọn
- Tự biến mất: KHÔNG (Phải bấm nút hoặc ngoài)
- Context: Bắt buộc dùng Activity Context
- Ví dụ: "Xác nhận xóa tài khoản?", "Thoát app?"

Toast trong Android: Cú pháp & Lưu ý Kỹ thuật

Phương thức thông báo đơn giản nhất nhưng cần chú ý gọi show() và Context

```
// Cú pháp chuẩn tạo Toast:
Toast.makeText(
    this,                    // Context (Activity hoặc Application)
    "Đăng nhập thành công!",    // Nội dung chuỗi thông báo
    Toast.LENGTH_SHORT         // Thời gian (SHORT ~2s, LONG ~3.5s)
).show()                     // BẮT BUỘC PHẢI GỌI show()!

// Tạo Extension Function tiện lợi trong Kotlin:
fun Context.showToast(message: String) {
    Toast.makeText(this, message, Toast.LENGTH_SHORT).show()
}

// Sử dụng ngắn gọn bất kỳ đâu trong Activity:
showToast("Chào mừng bạn quay trở lại!")
```

Lỗi kinh điển cần tránh:

- Quên gọi hàm '.show()': Toast được tạo nhưng không bao giờ xuất hiện lên màn hình!
- Từ Android 11 trở lên, hệ thống hạn chế việc tùy biến Custom View trên Toast để đảm bảo bảo mật.
- Không lạm dụng Toast liên tục vì các thông báo sẽ bị xếp hàng và kéo dài thời gian hiển thị gây khó chịu.

Snackbar Hiện đại: Tích hợp Nút Thao tác Action

Thông báo linh hoạt gắn với View cha, cho phép người dùng hoàn tác thao tác

```
import com.google.android.material.snackbar.Snackbar

// Tạo Snackbar gắn vào CoordinatorLayout hoặc Root View
val snackbar = Snackbar.make(
    binding.root,
    "Đã xóa 1 sinh viên khỏi danh sách",
    Snackbar.LENGTH_LONG
)

// Đính kèm nút Action Hoàn tác
snackbar.setAction("HOÀN TÁC") {
    // Logic khôi phục dữ liệu sinh viên vừa xóa
    restoreStudent()
}

// Tùy biến màu chữ nút Action
snackbar.setActionTextColor(Color.YELLOW)
snackbar.show()
```

Ưu điểm của Snackbar:

• Tham số đầu tiên là một View (thường dùng binding.root hoặc CoordinatorLayout).

• Nếu màn hình có FloatingActionButton (FAB), Snackbar sẽ tự động đẩy FAB lên trên rất chuyên nghiệp.

• Người dùng có thể vuốt ngang (swipe) để tắt Snackbar ngay lập tức mà không cần đợi hết giờ.

AlertDialog: Hộp thoại Xác nhận Quyết định

Sử dụng MaterialAlertDialogBuilder để đảm bảo giao diện đồng bộ Material 3

```
import com.google.android.material.dialog.MaterialAlertDialogBuilder

fun showConfirmDeleteDialog(onConfirm: () -> Unit) {
    MaterialAlertDialogBuilder(this)
        .setTitle("Xác nhận xóa")
        .setMessage("Hành động này không thể hoàn tác. Bạn chắc chắn muốn
xóa?")
        .setIcon(R.drawable.ic_warning)
        .setPositiveButton("Xóa vĩnh viễn") { dialog, _ ->
            onConfirm()
            dialog.dismiss()
        }
        .setNegativeButton("Hủy bỏ") { dialog, _ ->
            dialog.dismiss()
        }
        .setCancelable(false) // Bắt buộc người dùng bấm nút
        .show()
}
```

Điểm cần lưu ý:

- Ưu tiên dùng 'MaterialAlertDialogBuilder' thay vì 'AlertDialog.Builder' cũ.
- Thuộc tính 'setCancelable(false)': Ngăn người dùng chạm ra ngoài để đóng hộp thoại.
- 3 loại nút hỗ trợ: Positive (Đồng ý), Negative (Từ chối) và Neutral (Trung lập/Để sau).

Cấu trúc Giao diện Form Đăng ký Tương tác

Mô hình tổ chức các trường nhập liệu, cơ chế báo lỗi tức thì và nút kích hoạt



Thời gian thực
Lắng nghe qua event doOnTextChanged --> kiểm tra tính hợp lệ ngay khi người dùng gõ phím

Báo lỗi với text input layout
Bật thuộc tính errorEnabled = true
Gán binding.tilEmail.error = 'Email không hợp lệ'

Xác nhận khi bấm nút Đăng ký ngay
Kiểm tra tổng thể toàn Form
Nếu tất cả hợp lệ, hiển thị Snackbar thành công

Phần 4

Thực hành Dự án Form Đăng ký

Xây dựng hoàn chỉnh ứng dụng Interactive Registration: ViewBinding, Real-time Validation và Snackbar Feedback

Thực hành (Phần 1): Thiết lập & Real-time Validation

Khởi tạo ViewBinding và xóa lỗi tức thời khi người dùng bắt đầu chỉnh sửa

```
class RegistrationActivity : AppCompatActivity() {
    private lateinit var binding: ActivityRegistrationBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityRegistrationBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setupRealtimeValidation()
        binding.btnRegister.setOnClickListener { handleRegister() }
    }

    private fun setupRealtimeValidation() {
        binding.etName.doAfterTextChanged { binding.tilName.error = null }
        binding.etEmail.doAfterTextChanged { binding.tilEmail.error = null }
        binding.etPass.doAfterTextChanged { binding.tilPass.error = null }
        binding.etPhone.doAfterTextChanged { binding.tilPhone.error = null }
        binding.cbTerms.setOnCheckedChangeListener { _, isChecked ->
            if (isChecked) binding.cbTerms.error = null
        }
    }
```

Phân tích kỹ thuật:

- Hàm 'setupRealtimeValidation()' gắn bộ lắng nghe cho từng trường nhập liệu.
- Bất cứ khi nào người dùng gõ phím mới, lỗi cũ của trường đó lập tức bị dọn sạch.
- Tạo cảm giác mượt mà, không gây ức chế cho người dùng.

Mục tiêu Dự án: Interactive Registration Form

Xây dựng màn hình đăng ký tài khoản đạt chuẩn trải nghiệm người dùng chuyên nghiệp

Các trường thông tin cần xử lý:

• 1. Họ và tên: Bắt buộc, không được để trống, tối thiểu 2 từ.

• 2. Email: Bắt buộc, phải đúng định dạng email chuẩn (chứa ký tự @ và tên miền).

• 3. Mật khẩu: Bắt buộc, tối thiểu 6 ký tự, có hỗ trợ nút bật/tắt ẩn hiện mật khẩu.

• 4. Số điện thoại: Bắt buộc, đúng định dạng số di động Việt Nam (10 chữ số).

• 5. CheckBox: Bắt buộc người dùng phải tích chọn 'Đồng ý với điều khoản sử dụng'.

Hành vi ứng dụng yêu cầu:

• Xóa lỗi ngay khi người dùng gõ ký tự mới vào ô bị báo lỗi.

• Khi bấm nút Đăng ký, kiểm tra toàn diện: Nếu có lỗi, highlight đỏ các ô sai; nếu hợp lệ, xuất Snackbar thành công kèm nút [RESET].

Thực hành (Phần 2): Xử lý Kiểm tra & Nộp Form

Kiểm tra tổng thể, đánh dấu lỗi đồng loạt và phản hồi thành công với Snackbar

```
private fun handleRegister() {
    val name = binding.etName.text?.toString()?.trim().orEmpty()
    val email = binding.etEmail.text?.toString()?.trim().orEmpty()
    val pass = binding.etPass.text?.toString().orEmpty()
    var isValid = true

    if (name.isBlank()) {
        binding.tilName.error = "Họ tên không được để trống"; isValid = false
    }
    if (!android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
        binding.tilEmail.error = "Email không hợp lệ"; isValid = false
    }
    if (pass.length < 6) {
        binding.tilPass.error = "Mật khẩu tối thiểu 6 ký tự"; isValid = false
    }
    if (!binding.cbTerms.isChecked) {
        binding.cbTerms.error = "Cần đồng ý điều khoản"; isValid = false
    }

    if (isValid) {
        Snackbar.make(binding.root, "Đăng ký thành công: $name", Snackbar.LENGTH_LONG)
            .setAction("LÀM MỚI") { resetForm() }.show()
    }
}
```

Hoàn thiện luồng UX:

- Biến cờ 'isValid' đảm bảo tất cả các ô sai đều được phát hiện cùng lúc.
- Nếu toàn bộ trường hợp lệ, hiển thị Snackbar thông báo thành công.
- Cung cấp nút 'LÀM MỚI' trong Snackbar để reset form về trạng thái ban đầu một cách tiện lợi.

Kinh nghiệm & Best Practices khi Xử lý Sự kiện & Form

Những nguyên tắc vàng giúp mã nguồn an toàn, giao diện mượt mà và dễ bảo trì

Những điều NÊN làm:

• Luôn dùng 'trim()' trước khi kiểm tra dữ liệu chuỗi để loại bỏ khoảng trắng vô tình.

• Sử dụng 'doAfterTextChanged' kết hợp 'TextInputLayout.error' để báo lỗi trực quan.

• Tách riêng các hàm kiểm tra Validation độc lập vào Object để dễ dàng viết Unit Test.

• Gán 'inputType' phù hợp trong XML (textEmailAddress, textPassword, phone) để hiển thị đúng bàn phím ảo.

Những điều TUYỆT ĐỐI TRÁNH:

• Không thực hiện tác vụ mạng (Network API) hoặc ghi Database trực tiếp trong OnClickListener.

• Không gán chuỗi thông báo cứng (hardcode) trong code; nên khai báo trong 'res/values/strings.xml'.

• Không dùng thuộc tính 'android:onClick' trong file XML vì không an toàn kiểu dữ liệu.

Tổng kết Lecture 5

Kiểm tra & Bài tập về nhà

ĐÃ LÀM CHỦ KIẾN THỨC:

- Kiến trúc hướng sự kiện và chu trình bắt sự kiện trong Android.
- Bắt sự kiện Click, Text Change (KTX) và Checked Change với ViewBinding.
- Kỹ thuật Validate dữ liệu: Empty check, Độ dài, Regex Email & Phone.
- Hiển thị thông báo lỗi chuyên nghiệp với TextInputLayout Material 3.
- Sử dụng đúng thời điểm giữa Toast, Snackbar (Action) và AlertDialog.

BÀI TẬP VỀ NHÀ & EXIT TICKET:

- Bài tập 1: Hoàn thiện ứng dụng Interactive Registration Form với đủ 5 trường.
- Bài tập 2: Thêm trường 'Nhập lại Mật khẩu' và kiểm tra khớp với Mật khẩu ban đầu.
- Bài tập 3: Tích hợp AlertDialog hỏi 'Bạn có chắc muốn làm mới form?' khi bấm nút Reset.
- Câu hỏi ôn tập: So sánh ưu nhược điểm giữa Toast và Snackbar trong trải nghiệm người dùng.